

16

الدوال: تعريفها واستعمالها

الدالة (function) هي كتلة من التعليمات التي تنفَّذ إجراءً ما، ويمكن، بعد تعريفها، إعادة استخدامها في أكثر من موضع. تجعل الدوال الشيفرة تركيبية (modular)، مما يسمح باستخدام نفس الشفرة مرارًا وتكرارًا.

تضم بايثون عددًا من الدوال المُضمَّنة الشائعة، مثل:

- `print()` والتي تطبع كائنًا في الطرفية،
- `int()` والتي تحوّل أنواع البيانات النصية أو العددية إلى أعداد صحيحة،
- `len()` التي تعيد طول كائن، وغيرها من الدوال.

سنتعلم في هذا الفصل كيفية تعريف الدوال، وكيفية استخدامها في البرامج.

1. تعريف دالة

لنبدأ بتحويل البرنامج الذي يطبع عبارة "مرحبًا بالعالم!" إلى دالة.

أنشئ ملفًا نصيًا جديدًا، وافتحه في محرر النصوص المفضل عندك، ثم استدع البرنامج `hello.py`. تُعرّف الدالة باستخدام الكلمة المفتاحية `def`، متبوعة باسم من اختيارك، متبوعًا بقوسين يمكن أن يحتويوا المعاملات التي ستأخذها الدالة، ثم ينتهي التعريف بنقطتين. في هذه الحالة، سنعرّف دالة باسم `hello()`:

```
def hello():
```

في الشفرة أعلاه، أعددنا السطر الأول من تعريف الدالة.

بعد هذا، سنضيف سطرًا ثانيًا مُزاحًا بأربع مسافات بيضاء، وفيه سنكتب التعليمات التي

ستنفّذها الدالة. في هذه الحالة، سنطبع العبارة "مرحبًا بالعالم!" في سطر الأوامر:

```
def hello():
    print("مرحبا بالعالم")
```

لقد أتممنا تعريف الدالتنا، غير أننا إن نَقَّذنا البرنامج الآن، فلن يحدث أيُّ شيء، لأننا لم نستدع الدالة؛ لذلك، سنستدعي الدالة بالشكل `hello()` خارج كتلة تعريف الدالة:

```
def hello():
    print("!مرحبا بالعالم")

hello()
```

الآن، لننقِّذ البرنامج:

```
python hello.py
```

يجب أن تحصل على المخرجات التالية:

```
!مرحبا بالعالم
```

بعض الدوال أكثر تعقيداً بكثير من الدالة `hello()` التي عرَّفناها أعلاه. على سبيل المثال، يمكننا استخدام الحلقة `for` والتعليمات الشرطية، وغيرها داخل كتلة الدالة. على سبيل المثال، تستخدم الدالة المُعرَّفة أدناه تعليمة شرطية للتحقق مما إذا كانت المدخلات الممرَّرة إلى المتغير `name` تحتوي على حرف علة (`vowel`)، ثم تستخدم الحلقة `for` للمرور (`iterate`) على الحروف الموجودة في السلسلة النصية `name`.

```
# تعريف الدالة names()
def names():
    # إعداد المتغير name وإحالة المدخلات عليه
    name = str(input('أدخل اسمك باللغة الإنجليزية:'))
```

```
# التحقق من أن name يحتوي حرف علة
if set('aeiou').intersection(name.lower()):
    print('اسمك يحوي حرف علة')
else:
    print('اسمك لا يحوي حرف علة')

# المرور على حروف name
for letter in name:
    print(letter)

# استدعاء الدالة
names()
```

تستخدم الدالة `names()` التي عرّفناها أعلاه تعليمة شرطية، وحلقة `for`، وهذا توضيح لكيفية تنظيم الشفرة البرمجية ضمن تعريف الدالة. يمكننا أيضًا جعل التعليمة الشرطية والحلقة `for` دالتين منفصلتين.

تعريف الدوال داخل البرامج يجعل الشفرة البرمجية تركيبية (modular)، وقابلة لإعادة الاستخدام، وذلك سيتيح لنا استدعاء نفس الدالة دون إعادة كتابة شيفرتها كل مرة.

2. المعاملات: تمرير بيانات للدوال

حتى الآن، عرّفنا دالة ذات قوسين فارغين لا تأخذ أيّ وسائط (arguments)، سنتعلم في هذا القسم كيفية تعريف المعاملات (parameters) وتمرير البيانات إلى الدوال.

المعامل (parameter) هو كيان مُسمّى يوضع في تعريف الدالة، ويعرّف وسيطًا (arguments) يمكن أن تقبله الدالة عند استدعائها.

دعنا ننشئ برنامجًا صغيرًا يأخذ ثلاثة معاملات `x` و `y` و `z`. سننشئ دالة تجمع تلك المعاملات وفق عدة مجموعات ثمّ تطبع حاصل جمعها.

```
def add_numbers(x, y, z):
    a = x + y
    b = x + z
    c = y + z
    print(a, b, c)

add_numbers(1, 2, 3)
```

مرّرنا العدد 1 إلى المعامل x ، و 2 إلى المعامل y ، و 3 إلى المعامل z . تتوافق هذه القيم مع المعاملات المقابلة لها في ترتيب الظهور.

يُجري البرنامج العمليات الحسابية على المعاملات على النحو التالي:

```
a = 1 + 2
b = 1 + 3
c = 2 + 3
```

تطبع الدالة أيضًا a و b و c ، وبناءً على العمليات الحسابية أعلاه، فإن قيمة a ستساوي العدد 3، و b ستساوي 4، و c ستساوي العدد 5. لننفذ البرنامج:

```
python add_numbers.py
```

سنحصل على المخرجات التالية:

```
3 4 5
```

المعاملات هي متغيرات تُعرّف ضمن جسم الدالة. يمكن تعيين قيم إليها عند تنفيذ التابع بتمرير وسائط إلى الدالة، إذ تُسَدّ الوسائط إليها تلقائيًا آنذاك.

3. الوسائط المسماة

تُستدعى المعاملات بحسب ترتيب ظهورها في تعريف الدالة، أما الوسائط المسماة (Keyword Arguments) فتُستخدم بأسمائها في استدعاء الدالة.

عند استخدام الوسائط المسماة، يمكنك استخدام المعاملات بأيّ ترتيب تريد، لأنّ مترجم بايثون سيستخدم الكلمات المفتاحية لمطابقة القيم مع المعاملات. سننشئ دالة تعرض معلومات الملف الشخصي للمستخدم، ونمرّر إليها المُعاملين `username` (سلسلة نصية)، و `followers` (عدد صحيح).

```
# تعريف دالة ذات معاملات
def profile_info(username, followers):
    print("Username: " + username)
    print("Followers: " + str(followers))
```

داخل تعريف الدالة، وضعنا `username` و `followers` بين قوسي الدالة `profile_info()` أثناء تعريفها. تطبع شفرة الدالة المعلومات الخاصّة بالمستخدم على هيئة سلسلة نصية باستخدام المعاملين المُمرّرين. الآن، يمكننا استدعاء الدالة وتعيين المعاملات:

```
def profile_info(username, followers):
    print("Username: " + username)
    print("Followers: " + str(followers))

# استدعاء الدالة مع تعيين المعاملات
profile_info("sammyshark", 945)

# استدعاء الدالة مع تمرير الوسائط المسماة إليها
profile_info(username="AlexAnglerfish", followers=342)
```

في الاستدعاء الأول للدالة، مَرَرْنَا اسم المستخدم sammyshark، وعدد المتابعين 945 بالترتيب الوارد في تعريف الدالة. أمَّا في الاستدعاء الثاني للدالة، فقد استخدمنا الوسائط المسماة، وعَيَّنَّا قيمًا للوسائط ويمكن عكس الترتيب إن شئنا. لننفذ البرنامج:

```
python profile.py
```

سنحصل على المخرجات التالية:

```
Username: sammyshark
Followers: 945
Username: AlexAnglerfish
Followers: 342
```

سنحصل في المخرجات على أسماء المستخدمين، وأعداد المتابعين لكلا المستخدمين.

يمكننا تغيير ترتيب المعاملات، كما في المثال التالي:

```
def profile_info(username, followers):
    print("Username: " + username)
    print("Followers: " + str(followers))

# تغيير ترتيب المعاملات
profile_info(followers=820, username="cameron-catfish")
```

عند تنفيذ البرنامج أعلاه، سنحصل على المخرجات التالية:

```
Username: cameron-catfish
Followers: 820
```

يحافظ تعريف الدالة على نفس ترتيب التعليمات في `print()`، لذلك يمكننا استخدام

الوسائط المسماة بأيّ ترتيب نشاء.

4. القيم الافتراضية للوسائط

يمكننا إعطاء قيم افتراضية لواحد أو أكثر من المعاملات. في المثال أدناه، سنعطي

للمعامل `followers` القيمة الافتراضية 1 لاستعمالها إن لم تُمرَّر هذه القيمة للدالة عند استدعائها:

```
def profile_info(username, followers=1):
    print("Username: " + username)
    print("Followers: " + str(followers))
```

الآن، يمكننا استدعاء الدالة مع تعيين اسم المستخدم فقط، وسُيُعَيَّن عدد المتابعين تلقائيًا

ويأخذ القيمة 1. لكن يمكننا تغيير عدد المتابعين إن شئنا.

```
def profile_info(username, followers=1):
    print("Username: " + username)
    print("Followers: " + str(followers))
```

```
profile_info(username="JOctopus")
profile_info(username="sammyshark", followers=945)
```

عندما ننفِّذ البرنامج باستخدام الأمر `python profile.py`، ستظهر المخرجات التالية:

```
Username: JOctopus
Followers: 1
Username: sammyshark
Followers: 945
```

تمرير قيم إلى المعاملات الافتراضية سيتخطى القيمة الافتراضية المعطاة في

تعريف الدالة.

5. إعادة قيمة

كما يمكن تمرير قيم إلى الدالة، فيمكن كذلك أن تنتج الدالة قيمة وتعيدها لمن استدعاها. يمكن أن تنتج الدالة قيمة، ويكون ذلك عبر استخدام التعليمة `return`؛ هذه التعليمة اختيارية، وفي حال استخدامها، فستُنهي الدالة مباشرةً عملها وتوقف تنفيذها، وتُمرّر قيمة التعبير الذي يعقبها إلى المُستدعي (caller). إذا لم يلي التعليمة `return` أي شيء، فسُتعيد الدالة القيمة `None`.

حتى الآن، استخدمنا الدالة `print()` بدلاً من `return` في دوالنا لطباعة شيء بدلاً من إعادته. لننشئ برنامجًا يعيد متغيرًا بدلاً من طباعته الآن.

سننشئ برنامجًا في ملف نصي جديد يسمى `square.py` يحسب مربع المعامل `x`، ويُحيل الناتج إلى المتغير `y`، ثم يعيده. سنطبع المتغير `result`، والذي يساوي ناتج تنفيذ الدالة `square(3)`.

```
def square(x):
    y = x ** 2
    return y

result = square(3)
print(result)
```

لننفذ البرنامج:

```
python square.py
```

سنحصل على المخرجات التالية:

مخرجات البرنامج هي العدد الصحيح 9 الذي أعادته الدالة وهو ما نتوقعه لو طلبنا من

بايثون حساب مربع العدد 3.

لفهم كيفية عمل التعليمة `return`، يمكننا تعليق التعليمة `return`:

```
def square(x):
    y = x ** 2
    # return y

result = square(3)
print(result)
```

الآن، لننفذ البرنامج مرة أخرى:

```
python square.py
```

سنحصل على الناتج التالي:

```
None
```

بدون استخدام التعليمة `return`، لا يمكن للبرنامج إعادة أي قيمة، لذلك تُعاد القيمة

الافتراضية `None`.

إليك مثال آخر، في برنامج `add_numbers.py` أعلاه، سنستبدل بالعلامة `return`

الدالة `print()`:

```
def add_numbers(x, y, z):
    a = x + y
    b = x + z
    c = y + z
    return a, b, c
```

```
sums = add_numbers(1, 2, 3)
print(sums)
```

خارج الدالة، أحلنا إلى المتغير sums نتيجة استدعاء الدالة بالوسائط 1 و 2 و 3 كما فعلنا أعلاه ثم طبعنا قيمته.

فلننقذ البرنامج مرة أخرى:

```
python add_numbers.py
```

والناتج سيكون:

```
(3, 4, 5)
```

لقد حصلنا على الأعداد 3 و 4 و 5 وهي نفس المخرجات التي تلقيناها سابقًا عندما استخدمنا الدالة print(). هذه المرة تمت إعادتها على هيئة صف لأنَّ التعبير المرافق للتعليلة return يحتوي على فاصلة واحدة على الأقل.

توقّف الدوال فورًا عندما تصل إلى التعليلة return، سواء أعادت قيمة، أم لم تُعد.

```
def loop_five():
    for x in range(0, 25):
        print(x)
        if x == 5:
            # إيقاف الدالة عند x == 5
            return
    print("This line will not execute.")

loop_five()
```

يؤدي استخدام التعليمة `return` داخل الحلقة `for` إلى إنهاء الدالة، وبالتالي لن يُنفَّذ السطر الموجود خارج الحلقة.

لو استخدمنا بدلاً من ذلك `break`، فسيُنَفَّذ السطر `print()` الأخير من المثال السابق. نعيد التذكير أنَّ التعليمة `return` تنهي عمل الدالة، وقد تعيد قيمة إذا أعقبها تعبير.

6. استخدام `main()` دالةً رئيسيةً

رغم أنَّه يمكنك في بايثون استدعاء الدالة في أسفل البرنامج وتنفيذها (كما فعلنا في الأمثلة أعلاه)، فإنَّ العديد من لغات البرمجة (مثل `C++` و `Java`) تتطلب وجود دالة رئيسية تدعى `main`. إنَّ تضمين دالة `main()`، وإن لم يكن إلزاميًا، يمكن أن يهيكل برامج بايثون بطريقة منطقية، فتضع أهم مكونات البرنامج في دالة واحدة. كما يمكن أن يجعل البرنامج أكثر مقروئية للمبرمجين غير البايثونيين.

سنبدأ بإضافة دالة `main()` إلى برنامج `hello.py` أعلاه. سنحتفظ بالدالة `hello()`، ثم

نعرّف الدالة `main()`:

```
def hello():
    print("مرحبا بالعالم")

def main():
```

ضمن الدالة `main()`، سندرج الدالة `print()`، والتي ستُعَلِّمنا بأننا في الدالة `main()`. أيضًا

سنستدعي الدالة `hello()` داخل `main()`:

```
def hello():
    print("مرحبا بالعالم")

def main():
    print("هذه هي الدالة الرئيسية")
    hello()
```

أخيرًا، في أسفل البرنامج، سنستدعي الدالة `main()`:

```
def hello():
    print("!مرحبا بالعالم")

def main():
    print(".هذه هي الدالة الرئيسية")
    hello()

main()
```

الآن يمكننا تنفيذ برنامجنا:

```
python hello.py
```

وسنحصل على المخرجات التالية:

```
.هذه هي الدالة الرئيسية
!مرحبا بالعالم
```

لما استدعينا الدالة `hello()` داخل `main()`، ثم نَقَدْنَا الدالة `main()` وحدها، فقد طُبِع النص "مرحبا بالعالم!" مرة واحدة فقط، وذلك عقب [السلسلة النصية](#) التي أخبرتنا بأننا في الدالة الرئيسية. (سنعمل الآن مع دوال مُتَعَدِّدة، لذلك من المستحسن أن تراجع نطاقات المتغيرات في [فصل المتغيرات](#) إن كنت قد نسيتها.)

إذا عرّفت متغيرًا داخل دالة، فلا يمكنك أن تستخدم ذلك المتغير إلا ضمن تلك الدالة. لذا، إن أردت استخدام متغير ما في عدة دوال، فقد يكون من الأفضل الإعلان عنه متغيرًا عامًا (global variable).

في بايثون، يعدّ `'__main__'` اسم النطاق الذي سننقذ فيه الشيفرة العليا (top-level code). عند تنفيذ برنامج من الدخل القياسي (standard input)، أو من سكربت، أو من سطر الأوامر، سيتم ضبط `__name__` عند القيمة `'__main__'`. لهذا السبب، اصطلح مطورو بايثون على استخدام الصياغة التالية:

```
if __name__ == '__main__':
    الشفرة التي سننقذ لو كان هذا هو البرنامج الرئيسي
```

هذه الصياغة تتيح استخدام ملفات بايثون إما:

- برامج رئيسية، مع تنفيذ ما يلي التعليمة `if`، أو
- وحدات عادية، مع عدم تنفيذ ما يتبع التعليمة `if`.

سننقذ الشفرة غير المُتضمنة في العبارة `if __name__ == '__main__':` عند التنفيذ.

إذا كنت تستخدم ملف بايثون كوحدة، فسُنقذ أيضًا الشيفرة البرمجية غير المُتضمنة في هذه العبارة عند استيراد ذلك الملف.

دعنا نوسّع البرنامج `names.py` أعلاه، لننشئ ملفًا جديدًا يسمى `more_names.py`. سنعلن

في هذا البرنامج عن متغير عام، ونعدّل الدالة `names()` الأصلية بشكل نقسم فيه التعليمات إلى دالتين منفصلتين.

ستتحقق الدالة الأولى (`has_vowel()`) مما إذا كانت السلسلة النصية `name` تحتوي على حرف علة (`vowel`). وتطبع الدالة الثانية (`print_letters()`) كل حرف من السلسلة النصية `name`.

```
# الإعلان عن متغير عام لاستخدامه في جميع الدوال
name = str(input('أدخل اسمك باللغة الإنجليزية:'))

# تعريف دالة للتحقق من أن name يحتوي حرف علة
def has_vowel():
    if set('aeiou').intersection(name.lower()):
        print('اسمك يحتوي حرف علة')
    else:
        print('اسمك لا يحتوي حرف علة')

# المرور على حروف name
def print_letters():
    for letter in name:
        print(letter)
```

بعد ذلك، دعنا نعرّف الدالة (`main()`) التي ستستدعي كلا من الدالة (`has_vowel()`)

والدالة (`print_letters()`).

```
# الإعلان عن متغير عام لاستخدامه في جميع الدوال
name = str(input('أدخل اسمك باللغة الإنجليزية:'))

# تعريف دالة للتحقق من أن name يحتوي حرف علة
def has_vowel():
    if set('aeiou').intersection(name.lower()):
```

```

    print('اسمك يحتوي حرف علة')
else:
    print('اسمك لا يحتوي حرف علة')

# المرور على حروف name
def print_letters():
    for letter in name:
        print(letter)

# تعريف الدالة main التي ستستدعي بقية الدوال
def main():
    has_vowel()
    print_letters()

```

أخيرًا، سنضيف العبارة: `if __name__ == '__main__':` في أسفل الملف. لقد وضعنا

جميع الدوال التي نودُّ تنفيذها في الدالة `main()`، لذا سنستدعي الدالة `main()` بعد الشرط `if`.

```

# الإعلان عن متغير عام لاستخدامه في جميع الدوال
name = str(input('أدخل اسمك:'))

# تعريف دالة للتحقق من أن name يحتوي حرف علة
def has_vowel():
    if set('aeiou').intersection(name.lower()):
        print('اسمك يحتوي حرف علة')
    else:
        print('اسمك لا يحتوي حرف علة')

# المرور على حروف name

```



```
def print_letters():
    for letter in name:
        print(letter)

# تعريف الدالة main التي ستستدعي بقية الدوال

def main():
    has_vowel()
    print_letters()

# تنفيذ الدالة main
if __name__ == '__main__':
    main()
```

يمكننا الآن تنفيذ البرنامج:

```
python more_names.py
```

سيعرض هذا البرنامج نفس المخرجات التي عرضها البرنامج `names.py`، بيد أن الشفرة هنا أكثر تنظيماً، ويمكن استخدامها بطريقة تركيبية (modular).

إذا لم ترغب في الإعلان عن الدالة `main()`، يمكنك بدلاً من ذلك إنهاء البرنامج كما يلي:

```
...
if __name__ == '__main__':
    has_vowel()
    print_letters()
```

يؤدي استخدام دالة رئيسية مثل `main()`، واستخدام العبارة

`if __name__ == '__main__':` إلى تنظيم الشيفرة البرمجية بطريقة منطقية، وجعلها أكثر

مقروئية وتراكيبة.

7. استخدام `*args` و `**kwargs`

تعدُّ المعاملات في تعريف الدوال كيانات مسماة تُحدَّد وسيطًا (argument) يمكن أن يُمرَّر إلى الدالة المُعرَّفة.

أثناء البرمجة، قد لا تدرك جميع حالات الاستخدام الممكنة للشيفرة، لذا قد ترغب في توسيع خيارات المبرمجين المستقبليين الذين سيستخدمون الوحدة التي طورتها. سنتعلم في هذا القسم كيفية تمرير عدد متغير من الوسائط إلى دالة ما باستخدام الصياغتين `*args` و `**kwargs`.

1. `*args`

في بايثون، يمكن استخدام الشكل أحادي النجمة `*args` معاملاً لتمرير قائمة غير محدَّدة الطول من الوسائط غير المسماة (non-keyworded argument) إلى الدوال. تجدر الإشارة إلى أنَّ النجمة (*) عنصر ضروري هنا، إذ رغم أنَّ الكلمة `args` متعارف عليها بين المبرمجين، إلا أنَّها غير رسمية.

لنلق نظرة على مثال لدالة تستخدم وسيطين:

```
def multiply(x, y):
    print (x * y)
```

عرَّفنا في الشيفرة أعلاه دالة تقبل وسيطين `x` و `y`، عندما نستدعي هذه الدالة، سنحتاج إلى تمرير عددين موافقين للوسيطين `x` و `y`. في هذا المثال، سنمرِّر العدد الصحيح 5 إلى `x`، والعدد الصحيح 4 إلى `y`:

```
def multiply(x, y):
    print (x * y)

multiply(5, 4)
```

عند تنفيذ الشفرة أعلاه:

```
python lets_multiply.py
```

سنحصل على المخرجات التالية:

```
20
```

ماذا لو قرّرنا لاحقاً حساب ناتج ضرب ثلاثة أعداد بدلاً من عددين فقط؟ إذا حاولت تمرير

عدد إضافي إلى الدالة، كما هو موضح أدناه:

```
def multiply(x, y):
    print (x * y)

multiply(5, 4, 3)
```

فسيتطرق الخطأ التالي:

```
TypeError: multiply() takes 2 positional arguments but 3 were
given
```

إذا شككت أنّك ستحتاج إلى استخدام المزيد من الوسائط لاحقاً، فالحل هو

استخدام `*args` معاملاً. سنزيل المُعاملين `x` و `y` من الشيفرة في المثال الأول، ونضع

مكانهما `*args`:

```
def multiply(*args):
    z = 1
    for num in args:
        z *= num
    print(z)
```

```
multiply(4, 5)
multiply(10, 9)
multiply(2, 3, 4)
multiply(3, 5, 10, 6)
```

عندما ننفِّذ هذه الشيفرة، سنحصل على ناتج استدعاءات الدالة أعلاه:

```
20
90
24
900
```

يمكننا باستخدام الوسيط `*args` تمرير أي عدد نحب من الوسائط عند استدعاء الدالة بالإضافة إلى كتابة شيفرة أكثر مرونة، وإنشاء دوال تقبل عددًا غير محدد مسبقًا من الوسائط غير المسماة.

ب. `**kwargs`

يُستخدم الشكل ذو النجمتين `**kwargs` لتمرير قاموس متغيّر الطول من الوسائط المسماة إلى الدالة المعرّفة. مرة أخرى، النجمتان `(**)` ضروريتان، فمع أنّ استخدام الكلمة `kwargs` متعارف عليه لدى المبرمجين، إلا أنّها غير رسمية.

يمكن أن تأخذ `**kwargs`، كما هو شأن `*args`، أيّ عدد من الوسائط التي ترغب في

تمريرها إلى الدالة يُدَّ أن `**kwargs` تختلف عن `*args` في أنَّها تستوجب تعيين أسماء المعاملات (keywords).

في المثال التالي، ستطبع الدالة الوسيط `**kwargs` الممرر إليها:

```
def print_kwargs(**kwargs):
    print(kwargs)
```

سنستدعي الآن الدالة ونمرّر إليها بعض الوسائط المسماة:

```
def print_kwargs(**kwargs):
    print(kwargs)

print_kwargs(kwargs_1="Shark", kwargs_2=4.5, kwargs_3=True)
```

لننقذ البرنامج أعلاه:

```
python print_kwargs.py
```

سنحصل على المخرجات التالية:

```
{'kwargs_3': True, 'kwargs_2': 4.5, 'kwargs_1': 'Shark'}
```

اعتمادًا على إصدار بايثون 3 الذي تستخدمه، فقد لا يكون القاموس مرتبًا. في بايثون 3.6 وما بعده، ستحصل على أزواج قيمة-مفتاح (key-value) مرتبة، ولكن في الإصدارات السابقة، سيكون ترتيب الأزواج عشوائيًا.

سُيُنشَأ قاموس يسمى `kwargs`، والذي يمكننا التعامل معه مثل أي قاموس عادي داخل الدالة.

لننشئ برنامجًا آخر لإظهار كيفية استخدام `**kwargs`. سننشئ دالة تطبع قاموسًا من الأسماء. أولاً، سنبدأ بقاموس يحوي اسمين:

```
def print_values(**kwargs):
    for key, value in kwargs.items():
        print("The value of {} is {}".format(key, value))

print_values(my_name="Sammy", your_name="Casey")
```

بعد تنفيذ البرنامج عبر الأمر التالي:

```
python print_values.py
```

سنحصل على ما يلي:

```
The value of your_name is Casey
The value of my_name is Sammy
```

قد لا تكون القواميس مرتّبة، لذلك قد يظهر الاسم Casey أولاً، وقد يظهر ثانيًا. سنُمرّر الآن وسائط إضافية إلى الدالة لنرى كيف يمكن أن تقبل `**kwargs` أيّ عدد من الوسائط:

```
def print_values(**kwargs):
    for key, value in kwargs.items():
        print("The value of {} is {}".format(key, value))

print_values(
    name_1="Alex",
    name_2="Gray",
    name_3="Harper",
    name_4="Phoenix",
    name_5="Remy",
```

```
name_6="Val"
)
```

إذا نفّذنا البرنامج الآن، فسنحصل على المخرجات التالية، والتي قد تكون غير مرتبة:

```
The value of name_2 is Gray
The value of name_6 is Val
The value of name_4 is Phoenix
The value of name_5 is Remy
The value of name_3 is Harper
The value of name_1 is Alex
```

يتيح لك استخدام `**kwargs` مرونةً كبيرةً في استخدام الوسائط المسماة. فعند

استخدامها، لن نحتاج إلى معرفة مسبقة بعدد الوسائط التي ستمرر إلى الدالة.

8. ترتيب الوسائط

عند الخلط بين عدة أنواع من الوسائط داخل دالة، أو داخل استدعاء دالة، يجب أن تظهر

الوسائط وفق الترتيب التالي:

- الوسائط العادية
- `*args`
- الوسائط المسماة
- `**kwargs`

عملياً، عند الجمع بين المعاملات العادية، والوسيطتين `*args` و `**kwargs`، فينبغي أن

تكون وفق الترتيب التالي:

```
def example(arg_1, arg_2, *args, **kwargs):
    ...
```

وعند الجمع بين المعاملات العادية والمعاملات المسماة و `*args` و `**kwargs`، ينبغي أن

تكون وفق الترتيب التالي:

```
def example2(arg_1, arg_2, *args, kw_1="shark",
             kw_2="blobfish", **kwargs):
    ...
```

من المهم أن تأخذ في الحسبان ترتيب الوسائط عند إنشاء الدوال حتى لا تتسبب في

إطلاق خطأ متعلق بالصياغة.

9. استخدام `*args` و `**kwargs` في استدعاءات الدوال

يمكننا أيضًا استخدام `*args` و `**kwargs` لتمرير الوسائط إلى الدوال. أولًا، دعنا ننظر إلى

مثال يستخدم `*args`:

```
def some_args(arg_1, arg_2, arg_3):
    print("arg_1:", arg_1)
    print("arg_2:", arg_2)
    print("arg_3:", arg_3)

args = ("Sammy", "Casey", "Alex")
some_args(*args)
```

في الدالة أعلاه، هناك ثلاثة معاملات، وهي `arg_1` و `arg_2` و `arg_3`. ستطبع الدالة كل هذه

الوسائط. بعد ذلك أنشأنا متغيرًا، وأحلنا عليه عنصرًا تكراريًا (في هذه الحالة، صف)، ثم مررنا

ذلك المتغير إلى الدالة باستخدام الصياغة النجمية (asterisk syntax).

عندما ننفذ البرنامج باستخدام الأمر `python some_args.py`، سنحصل على

المخرجات التالية:

```
arg_1: Sammy
arg_2: Casey
arg_3: Alex
```

يمكننا أيضًا تعديل البرنامج أعلاه، واستخدام قائمة. سندمج أيضًا `*args` مع

وسيط مسمى:

```
def some_args(arg_1, arg_2, arg_3):
    print("arg_1:", arg_1)
    print("arg_2:", arg_2)
    print("arg_3:", arg_3)

my_list = [2, 3]
some_args(1, *my_list)
```

إذا نفذنا البرنامج أعلاه، فسنحصل على المخرجات التالية:

```
arg_1: 1
arg_2: 2
arg_3: 3
```

وبالمثل، يمكن استخدام الوسائط المسماة `**kwargs` لاستدعاء دالة.

سننشئ متغيرًا، ونسند إليه قاموسًا من 3 أزواج مفتاح-قيمة (سنستخدم `kwargs` هنا،

ولكن يمكنك تسميته ما تشاء)، ثم نُمرّره إلى دالة ذات 3 وسائط:

```
def some_kwargs(kwarg_1, kwarg_2, kwarg_3):
    print("kwarg_1:", kwarg_1)
    print("kwarg_2:", kwarg_2)
    print("kwarg_3:", kwarg_3)

kwargs = {"kwarg_1": "Val", "kwarg_2": "Harper", "kwarg_3":
"Remy"}
some_kwargs(**kwargs)
```

عند تنفيذ البرنامج أعلاه باستخدام الأمر `python some_kwargs.py`، سنحصل على

المخرجات التالية:

```
kwarg_1: Val
kwarg_2: Harper
kwarg_3: Remy
```

10. خلاصة الفصل

الدوال هي كتل من التعليمات البرمجية التي تُنفَّذ إجراءات معيّنة داخل البرنامج، كما تساعد على جعل الشيفرة تركيبية، وقابلة لإعادة الاستخدام بالإضافة إلى تنظيمها وتسهيل قراءتها. (لمعرفة المزيد حول كيفية جعل الشفرة تركيبية، يمكنك الرجوع إلى فصل الوحدات).

يمكنك استخدام الصياغتين `*args` و `**kwargs` الخاصتين ضمن تعريف وسائط الدوال لتمرير أي عدد تشاء من الوسائط إليها، إذ يُستحسن استخدام `*args` و `**kwargs` في المواقف التي تتوقع أن يظل فيها عدد المدخلات في قائمة الوسائط صغيرًا نسبيًا. وضع في ذهنك أن استخدام `*args` و `**kwargs` يحسّن مقروئية الشيفرة، ويسهّل على المبرمجين، ولكن ينبغي استخدامهما بحذر.